Day 48 – OData Adapter

Class date: 5th September 2026 | Module: Adapters (OData), Query Options & Pagination

Correction to your handwritten notes: you wrote **"It's open source → there's no schema definition for this."** This isn't quite right, and it actually contradicts your own very next lines about the EDMX file. OData services **do** have a formal schema – it's just retrieved dynamically rather than shared as a separate static file like a WSDL. Appending **$metadata** to the Service Root URL returns exactly this schema, as an **EDMX document** (covered correctly in your own next section). "Open" in "Open Data Protocol" refers to OData being an open, published standard (like an open specification anyone can implement), not to the service having no schema at all.

1. What is OData? (start here first)

OData stands for **Open Data Protocol**. The real question worth asking is: what is OData actually *for*, beyond just being "another API type"?

OData is itself built on top of REST-based APIs – it is not a separate, unrelated concept from REST. The one real difference is: **OData adds a standardized set of advanced query capabilities** on top of plain REST, which a standard/custom REST API doesn't have built in.

Advanced capabilities, compared:

Standard REST API	OData (built on REST)
Sort, filter, select-specific-fields are **not** directly/uniformly supported – each API would need custom-built logic for this.	Sort (**$orderby**), filter (**$filter**), and select-specific-fields (**$select**) are all **standardized, built-in query capabilities** available on any OData service.

Example used in class: the Northwind API exists in two forms – a plain REST-based API call, and a dedicated **Northwind OData** service. The underlying data is the same; OData just layers standardized query capabilities on top.

Key takeaway: OData is not a separate concept from REST – it's **built on REST**. The only real reason to prefer OData over a plain REST API is its **additional standardized query capabilities.**

2. OData URI – how it's structured

Reference used in class: the official **odata.org** documentation → OData 2.0 → Overview → **URI Conventions** page.

Main concept: a **Host name and Port** are the foundation of any API call – both are mandatory. Example: **https (protocol), host, port** (a common combination being port 443 for HTTPS).

General OData URI structure:

```
https://host:port//?
```

- **OData consists of different Resources or Entities** – e.g. a Calculator-style WSDL might expose *operations* like Add, Subtract, Multiply; an OData service instead exposes *Entities* such as Employees, Orders, Products.
- Whenever a **?<query option>** exists in the URL, that URL is called a **Query URI**.

Real example used in class (Northwind's actual public OData V2 demo service):

```
https://services.odata.org/V2/northwind/northwind.svc/Employees host ->
services.odata.org Service Root URL -> /V2/northwind/northwind.svc Resource/Entity
-> Employees
```

Demo: Postman was used to demonstrate placing a **"?"** in the URL to begin adding query options.

3. Why OData over plain HTTP? – WSDL vs. EDMX prerequisite

Just like the **SOAP Adapter** needs a **WSDL file** as its prerequisite (the schema definition listing what operations a service has), the **OData Adapter** needs an **EDMX file** as its prerequisite.

What is an EDMX file? It is the **schema definition** for an OData service – the complete schema showing what **Entities** exist and what **Fields** each Entity has.

How the EDMX file is generated/retrieved: take the Service Root URL and append **$metadata** to it – this returns the schema definition (EDMX) file directly from the service itself, live.

```
https:///$metadata -> returns the schema definition (EDMX) file
```

Demo shown: copying the EDMX file of the Northwind API this way. Key structural terms inside it:

- **Entity Container** – the list of all Entities exposed by the service.
- **Employee (an Entity)** – itself contains a list of Fields (e.g. EmployeeID, FirstName, LastName, etc.).
- **Order Details (another Entity)** – also has its own list of Fields.

4. Standard OData Query Options

OData (built on REST, remember) provides **8 standard query options:**

#	Option	Purpose
1	$select	Return only the specified fields, instead of every field on the Entity.
2	$expand	Also pull in related/linked Entities in the same call (covered as "pending" – to be detailed in a future class).
3	$filter	Filter records based on a condition (covered as "pending" – to be detailed in a future class).
4	$orderby	Sort the returned data, ascending or descending.
5	$top	Fetch only the top N records.
6	$skip	Skip a specified number of records (used together with $top for pagination).
7	$inlinecount	Return the total count of all records available for that Entity/API, in addition to the data.
8	$format	Return the result in a specific format, e.g. JSON.

Standard vs. Custom query options: Standard OData query options always start with a **$** (dollar) symbol, e.g. **?$select=...**. **Custom** query options (application-specific parameters defined by that particular service) do **not** use the $ symbol, e.g. **?custom=...**.

5. $select – fetching only required fields

If you call an OData API without any query option, you get the **complete result** – every field, on every record.

Real-world need: suppose you only need **2 fields**, but the Entity actually has **40 fields**. Pulling all 40 every time is wasteful. **OData is useful here**, since a plain REST/HTTP call without OData-style query support is **not recommended** for this – you'd have to build custom filtering logic yourself in the iFlow after the fact.

Example: if only **OrderID** and **ProductID** are required from the Order Details Entity, use the Select query:

```
/Order Details?$select=OrderID,ProductID
```

6. $orderby – sorting the data

Example: sort data using ProductID (e.g. raw order: 11, 42, 72, 14, 2 – unsorted).

```
?$orderby=ProductID desc (The default sort direction, if not specified, is
Ascending.) ?$orderby=OrderID desc
```

7. $top, $skip, $inlinecount, $format

$top – fetches the top N number of records.

```
?$top=200
```

Multiple query options can be combined using **&**:

```
?$top=200&$skip=50
```

$inlinecount – used to get the **count of all records** available for that particular API, in addition to the actual data. Options: **allpages** (include the count) or **none** (don't include it). In the response, this appears as an element like **<m:count>2155</m:count>**, showing the total records available.

*(Note: **$inlinecount** is the OData V2 syntax used in this demo. OData V4 uses a different, related system query option, **$count=true**, for the same purpose – worth knowing if you encounter a V4 service later.)*

$format – returns the result in the required format. **OData V2** defaults to **XML/Atom** format, and you explicitly request JSON with **$format=json**. **OData V4** defaults to **JSON** directly (and, unlike V2, does not support XML as a response format at all).

```
?$format=json
```

8. Pagination – why $top and $skip matter most

$top and $skip are the most important topic here – this is what implements **pagination**: with OData, you typically do **not** fetch all records in one shot; you fetch them in manageable pages.

Pagination concept, as illustrated in class (e.g. Google Search results, where results are split across numbered pages):

```
Page 1 -> records 1 to 100 Page 2 -> records 101 to 300 (i.e. the next batch)
(General API pagination pattern shown in class.)
```

SOAP vs. OData – "heavyweight" vs. "lightweight," as compared in class: if a SOAP API call returns 10,000 records, a single SOAP call will typically fetch **all 10,000** records at once (no native pagination in

the contract). With OData, a call for 10,000 records can instead be broken into smaller batches (e.g. 100, 200, 1000 at a time) using $top/$skip – each call only provides the requested slice, keeping payloads smaller and more manageable.

9. Syntax quick reference

Query Option	Syntax
Select	host:port/ServiceURL/EntityName?$select=ColumnName (e.g. ProductID,OrderID)
OrderBy	?$orderby=ColumnName asc/desc
Top	?$top=200
Skip	?$top=200&$skip=100
InlineCount	?$inlinecount=allpages / none
Format	?$format=json

$expand and $filter syntax are marked as pending in your notes – these will be covered with full detail (and worked examples) in an upcoming class.

Quick-recall summary

- **OData** = Open Data Protocol, built **on top of REST**, adding standardized query capabilities (sort/filter/select) that plain REST doesn't have natively.
- URI structure: **host:port/Service Root URL/Resource(or Entity)?query option.**
- OData Adapter's prerequisite: **EDMX file** (schema), retrieved via **Service Root URL + $metadata** – this is a real schema, not "no schema."
- 8 standard query options: **$select, $expand, $filter, $orderby, $top, $skip, $inlinecount, $format** – all start with $; custom query params don't.
- **$top/$skip** = pagination, the most practically important pair; **$inlinecount=allpages** returns total record count (V2); V4 uses **$count=true** instead.
- OData V2 default format = XML/Atom (use $format=json for JSON); OData V4 default and only supported format = JSON.